

Task 2 Part C: Counting Bloom Filter Analysis

Issues with Deletion in Counting Bloom Filters

The fundamental challenge with deletion in counting Bloom filters stems from **hash collision sharing** and the probabilistic nature of the data structure. Several critical issues emerge:

1. Shared Counter Problem Multiple distinct items can hash to the same bucket combinations, causing their counters to overlap. When deleting an item, we cannot distinguish whether a counter value represents:

- Only the item being deleted
- The item being deleted plus other items
- Only other items (if we mistakenly believe an item exists due to false positives)

This ambiguity means deletion can inadvertently affect the representation of other legitimate items in the filter.

2. False Negative Creation Unlike standard Bloom filters which guarantee no false negatives, counting Bloom filters with deletion can create them. If we delete an item that shares counters with existing items, we might reduce counters below their true values, causing subsequent queries for legitimate items to return "definitely not present" when they should return "possibly present."

3. Counter Overflow and Underflow With c bits per counter, each bucket is limited to counts $0 \leq \text{count} \leq 2^c - 1$. When multiple items hash to the same buckets:

- **Overflow:** Adding items beyond $2^c - 1$ corrupts the count
- **Underflow:** Deleting non-existent items or over-deleting can reduce counters below zero

Complexity Analysis

Variables:

- m = number of buckets in the filter
- k = number of hash functions
- c = bits per counter (variable parameter)
- n = number of items stored

Space Complexity

- **Standard Bloom Filter:** $O(m)$ bits (1 bit per bucket)
- **Counting Bloom Filter:** $O(m \times c)$ bits
- **Overhead factor:** c times larger than basic filter

Time Complexity

Insertion: $O(k)$

- Compute k hash values: $O(k)$
- Update k counters: $O(k)$ bucket updates

- Bit manipulation for c-bit counters is $O(1)$ with bitwise operations
- **Note:** While individual counter updates involve c-bit arithmetic, this is typically $O(1)$ for reasonable values of c

Deletion: $O(k)$

- Compute k hash values: $O(k)$
- Decrement k counters: $O(k)$
- Additional underflow checking: $O(k)$

Query: $O(k)$

- Compute k hash values: $O(k)$
- Read k counter values: $O(k)$
- Find minimum among k values: $O(k)$

Impact Summary

The transition from Part A to Part B multiplies space complexity by factor c while maintaining the same asymptotic time complexities. However, the practical overhead includes additional logic for counter management, overflow detection, and the fundamental loss of the "no false negatives" guarantee that makes standard Bloom filters reliable.

Task 2 Part E: Dynamic Bloom Filter Analysis

Complexity Changes from Part A to Part D

The evolution from a basic Bloom filter to a dynamic Bloom filter addresses scalability limitations while introducing new complexity considerations. The key innovation is replacing a single fixed-size filter with a linked list of counting Bloom filters that can grow dynamically.

Space Complexity Evolution

Part A (Basic): $O(m)$ bits **Part D (Dynamic):** $O(s \times m \times c)$ bits

Where:

- s = number of active Bloom filter layers
- m = buckets per individual filter
- c = bits per counter (4 bits in our implementation)

Additional tracking overhead: $O(s \times \text{capacity})$ for bird name storage, plus $O(s)$ for metadata (first_bird, last_bird pointers per filter).

Time Complexity Analysis

Insertion: $O(k)$

- Only inserts into the current active filter

- Same as Part A: compute k hashes and update k positions
- Filter creation when capacity reached is $O(1)$ amortized

Query: $O(k \times s)$ worst case, $O(k + s)$ average case

- Must potentially check all s filters
- With alphabetical ordering optimization: check only relevant filters based on bird name ranges
- Best case: $O(k)$ when bird falls clearly within one filter's range

Deletion: $O(k + s)$ implementation Our $O(k + s)$ deletion is achieved through:

1. **$O(s)$ Filter Location:** Use `birdNames` arrays to find which filter contains the target bird
2. **$O(k)$ Hash Computation:** Calculate k hash values once for the located filter
3. **$O(k)$ Counter Updates:** Decrement k counters in the identified filter
4. **$O(1)$ Metadata Update:** Remove bird from tracking arrays

This avoids the naive $O(k \times s)$ approach of checking every filter.

Implementation Strategy for Efficiency

Tracking Optimization: Each filter maintains:

- `birdNames[]` array for $O(s)$ membership testing
- `first_bird/last_bird` pointers for range-based filtering
- Current capacity counters for overflow detection

Reasonable Constant Assumptions: Following hash table analysis conventions:

- `BUCKET_SIZE` ($c = 4$) treated as constant
- `NUM_HASHES` (k) treated as small constant
- Individual filter capacity treated as reasonable constant

Performance Trade-offs

Advantages over Part A:

- Unlimited capacity growth
- Maintained $O(k)$ insertion performance
- Efficient deletion capability

Costs:

- Increased space overhead for tracking (factor of s)
- Query performance degrades with number of filters
- More complex implementation and memory management

The dynamic approach successfully removes the fixed-size limitation of Part A while keeping core operations efficient through careful tracking and optimization strategies.